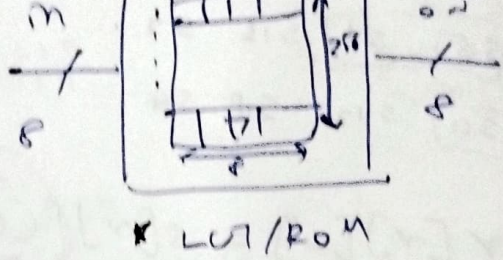


AES (summary)

(not clocked)

1) S-box :-

$$256 = 2^8$$



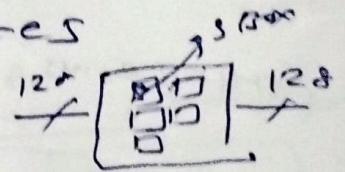
assign $o[j] = sbox[m]$;

$$00 \rightarrow 63$$

$m \quad op$

2) SUBBYTES: $\rightarrow$ 16 sboxes

$$\frac{128}{8} = 16$$



$$[127:0] \Rightarrow \frac{127:0}{i=15}$$

$$15:0 \quad 7:0$$

$i=0$

$$so \Rightarrow [8*i + 7 : 8*i] \quad (i=0 \rightarrow 15)$$

3) SHIFT ROWS: $[s_{15} \dots s_0]$

$MSB \quad LSB$

stores columnwise (s)

$$\begin{bmatrix} s_{15} \\ s_{14} \\ s_{13} \\ s_{12} \end{bmatrix} \rightarrow \begin{bmatrix} s_{11} \\ s_{10} \\ s_9 \\ s_8 \end{bmatrix} \rightarrow \begin{bmatrix} s_7 \\ s_6 \\ s_5 \\ s_4 \end{bmatrix} \rightarrow \begin{bmatrix} s_3 \\ s_2 \\ s_1 \\ s_0 \end{bmatrix}$$

$r=0$
 $r=1$
 $r=2$
 $r=3$

$$c \rightarrow 32 \text{ bits}$$

$$r \rightarrow 8 \text{ bits}$$

$$16 \times 16 \times 16 \times 16$$

$$s[r][c] = [127 - (c * 32) - (r * 8) - : p]$$

$$r=1, c=0 \quad (127 - (0 * 32) - (1 * 8) - : 15)$$

$$= [119 : 112] \rightarrow [119 : 112]$$

Shift Left

- row 0
- row 1
- row 2
- row 3

- $b_5(0)$
- $b_5(1)$
- $b_5(2)$
- $b_5(3)$

- $c=0$
- $c=1$
- $c=2$
- $c=3$

(No shift)
 $\Rightarrow$ row no

Final shifted matrix: - (sr)

S15 S11 S7 S3

S10 S6 S2 S14

S5 S1 S13 S9

S0 S12 S8 S4

Log + shift

$$sr[r][c] = s[r][\underline{(c+r) \% 4}]$$

Ex:

$$sr[1][0] = s[1][(1+0) \% 4] \\ = s[1][1] = \text{S10}$$

(SHIFT rows is just WIRES CONNECTED IN DIFFERENT MANNER)

"wire[7:0] s[0:3][0:3];"

4) MIX COLUMNS! *

1 column at a time

$$\begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \times \begin{bmatrix} S15 \\ S14 \\ S13 \\ S12 \end{bmatrix} \dots \begin{bmatrix} S3 \\ \vdots \\ S0 \end{bmatrix}$$

$$\begin{bmatrix} \boxed{} \end{bmatrix} \rightarrow 2 \times S15 \oplus 3 \times S14 \\ \oplus 1 \times S13 \oplus S12$$

(XOR not (+))

$GF(2^8)$ arithmetic

So we need

- x1 -
- x2 -
- x3 ✓

treats bts as

8 power polynomials

(x + 1)

*2 rule

(For AES)

if (MSB) = 0 just left shift

if (MSB = 1) left shift $\wedge$ 1B

$\Rightarrow$ 1B
00011011
1

To prevent overflow

$(u \ll 1) \wedge (8'h1B)$

$(u \ll 1)$ use this

x3:- $3 * u = ((2 * u) \oplus (u))$

OP
byte depends
on all
4
input
bytes

mixcol = (31:0) i/p col

31:24
23:16
15:8
7:0

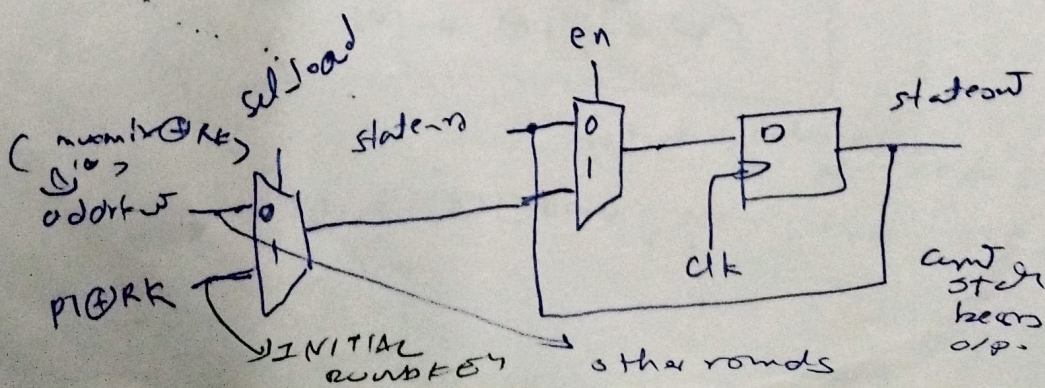
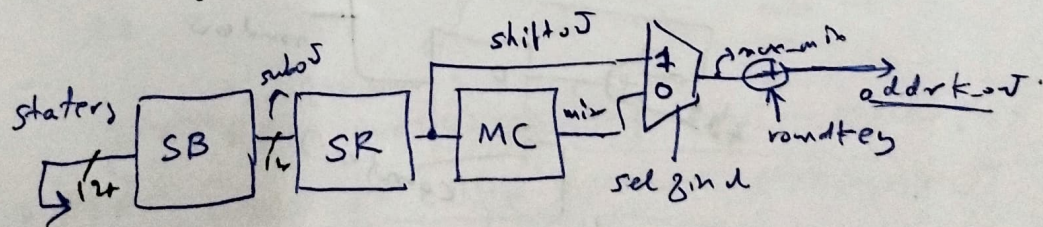
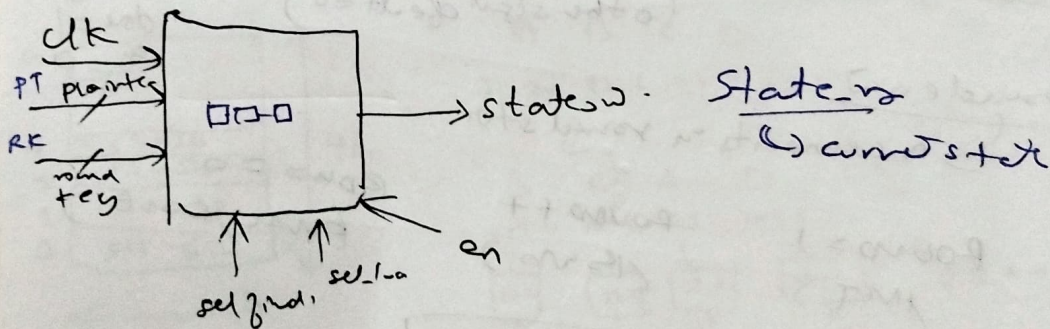
(DIFFUSION)

$s_0 \rightarrow x_1$
 $t_0 \rightarrow x_2$
 $s_0 \wedge t_0 \rightarrow x_3$

5) ADDROUND KEY $\rightarrow$ just $\oplus$ with Round key

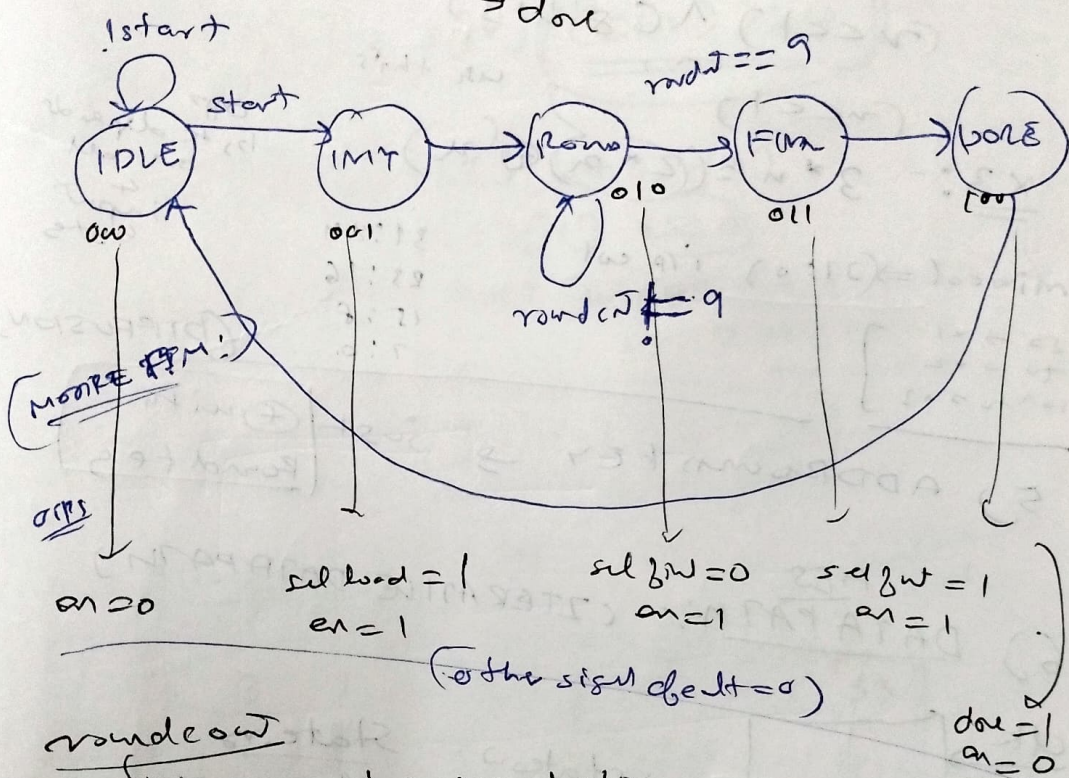
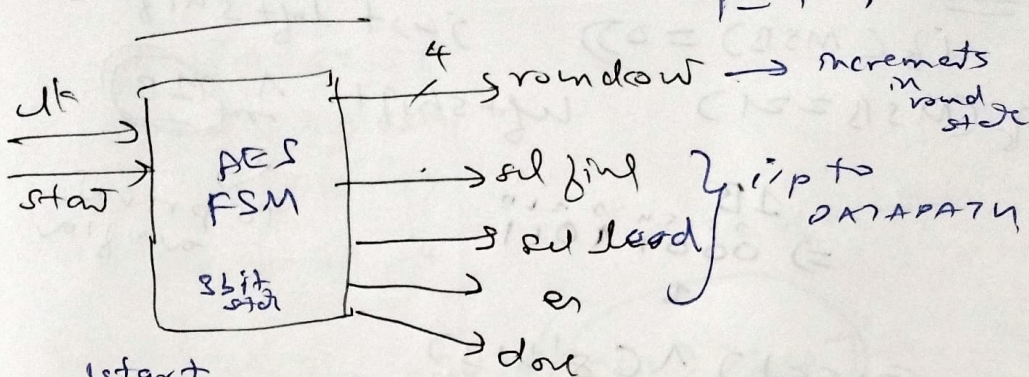
AES

6) DATAPATH:- (ITERATIVE DATAPATH)



7) AES FSM: -

1 100 1010
1-9, 10



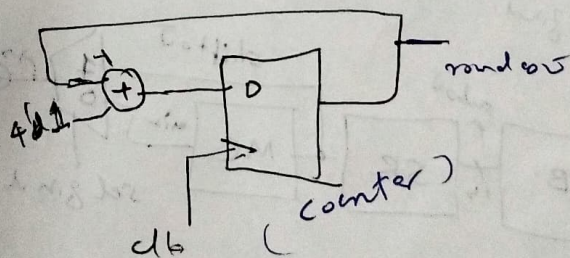
roundcnt

increments in round state

Round = 1
(INIT)

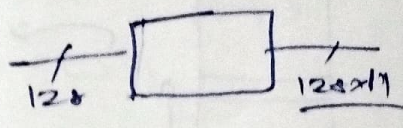
Round ++
(Round)

Round == 0
(FINAL, DONE)



8) AES KEYSCHEDULE - / KEY EXPANSION.

"11 diff round keys"
 (key 0) ... key 10
 10 rounds, 11 round key
 (10 for rounds, 1 for initial round.)



$11 \times 128 \Rightarrow 1408$ (1407:0) rkey-f(0)

stores all round keys together

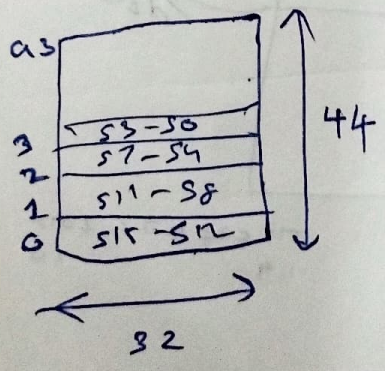
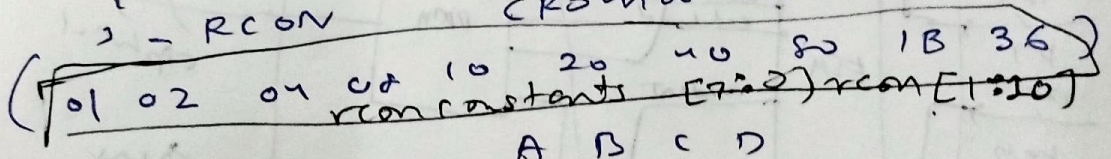
3264 words

$\frac{128}{32} = 4$ words per byte

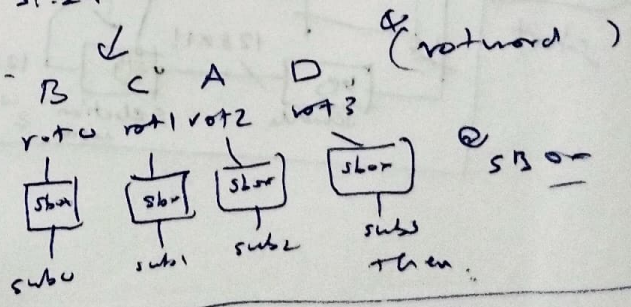
11 keys $\Rightarrow 4 \times 11 = 44$ words

Transformation, ~~Subword~~:

- 1 - Rotword (byte shifting)
- 1 - Subword (uses SBox)
- 1 - RCON (round constants)



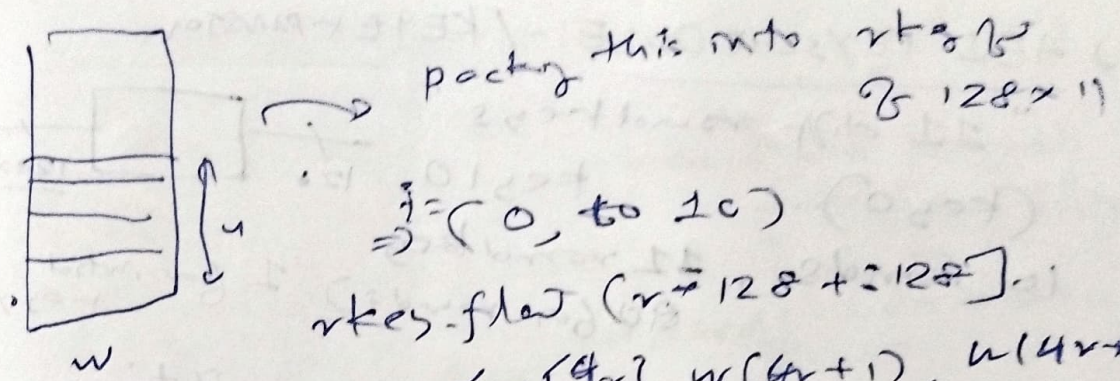
A B C D
 31:24 23:16 15:8 7:0



(non first words)
 • REST words $\Rightarrow w_i = w_{i-4} \oplus w_{i-1}$
 • FIRST WORD OF EACH ROUND FORMULAE
 $w_i = w_{i-4} \oplus g(w_{i-1})$

rotword, subword, xor rcon

$w_4 = w_0 \oplus (S(B) \oplus RCON, S(C), S(D), S(A))$
 $w_5 = w_1 \oplus w_4$
 $w_6 = w_2 \oplus w_5$
 $w_7 = w_3 \oplus w_6$



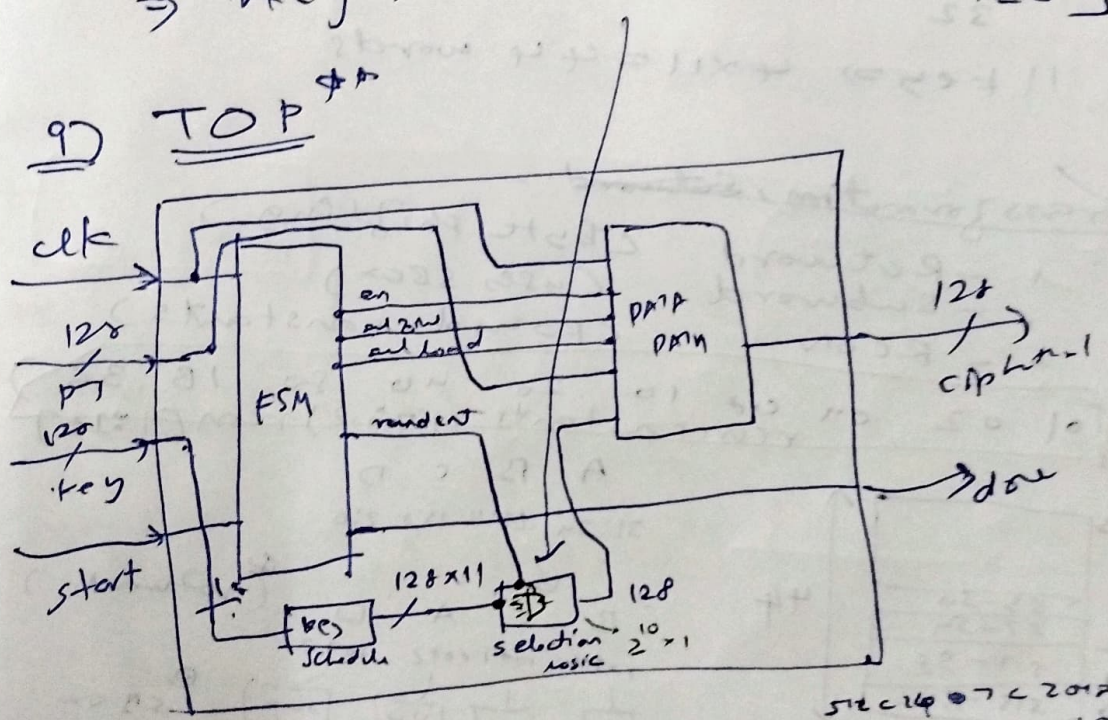
$i = (0, \text{to } 10)$

rkey-flat ($r = 128 + i \cdot 128$)
 $= \{w[4r], w(4r+1), w(4r+2), w(4r+3)\}$

WHILE USING IN TOP;

$\Rightarrow$ rkey-flat [$\text{roundent} \cdot 128 + 128$];

9) TOP



5/26/07 9 10

UART ^(i/p) ← display (7 seg) _(o/p)

FILE HANDLING

• (I/O HANDLING FOR AES)

UART RX: